Subiect A

1. $y = (a - (20 \cdot b) - 25) \cdot (c - (20 - d) \cdot 25)$

i) arbore sintactic

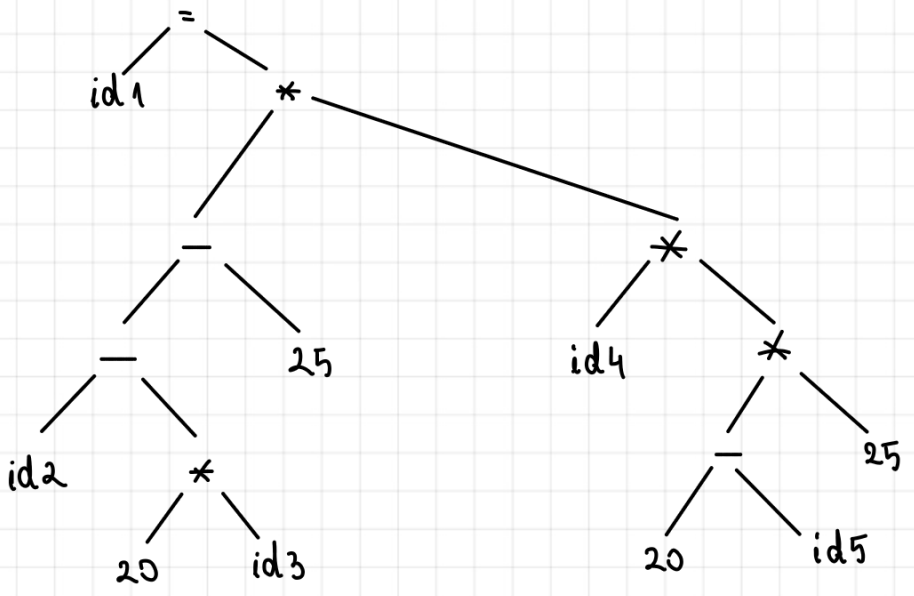
Identificatori

id1	y
id2	a
id3	b
id4	c
id5	d

Operatori: =, −, ·

Întregi: 20, 25

Analiză lexicală: id1 = (id2 − (20 · id3) − 25) · (id4 · (20 − id5) · 25)



ii) codul intermediar

temp1 = inttoreal (20)

temp2 = id3 * temp1

temp3 = id2 − temp2

temp4 = inttoreal (25)

temp5 = temp3 − temp4

temp6 = temp1 − id5

temp7 = temp6 * temp4

temp8 = id4 * temp7

temp9 = temp5 * temp8

id1 = temp9

iii) cod optimizat

temp 1 = 20 * id 3

temp 2 = id 2 - temp 1

tem 3 = temp 2 * 25

temp 4 = 20 - id 5

temp 5 = temp 4 * 25

temp 6 = id 4 * temp 5

id 1 = temp 3 * temp 6

iv) cod final

(id 3) $\longrightarrow R_1$

20 * R_1 $\longrightarrow R_1$

(id 2) $\longrightarrow R_2$

$(R_2) - (R_1) \longrightarrow R_1$

$(R_1) - \#25 \longrightarrow R_1$

(id 5) $\longrightarrow R_3$

$\#20 - (R_3) \longrightarrow R_3$

$\#25 * (R_3) \longrightarrow R_3$

(id 4) $\longrightarrow R_4$

$(R_4) * (R_3) \longrightarrow R_4$

$(R_1) * (R_4) \longrightarrow R_1$

$(R_1) \longrightarrow$ id 1

$(20 - d) * 25$

2.

2. Consideră următorul cod:

```
int tab[3];
void p2(int &x, int &y)
{
   x=y*y;
   y=x*x;
}
void p1(int &x, int &y)
{
   p2(y,x);
   x=y*y;
   y=x*x;
}
void main()
{
   tab[0]=3;
   tab[2]=4;
   p1(tab[0],tab[2]);
}
```

Care sunt valorile și cum au fost obținute pentru elementele tabloului tab după apelul procedurilor, dacă transmiterea parametrilor se face prin referință (&)?

tab

0	1	2
3	0	4

→ variabilă globală

p1(tab[0] , tab[2]) ⟶ p1(3,4)

tab[2]; tab[0]

p2(4 , 3) ⟶ { x = 3*3 = 9 → tab[2]
y = tab[2] * tab[2] = 9*9 = 81 → tab[0]

după p2 → tab[2] = 9 , tab[0] = 81

x = tab[2] * tab[2] = 9*9 = 81 → tab[0]

y = tab[2] * tab[2] = 81*81 = 6561 → tab[2]

tab

0	1	2
81	0	6561

(3.)

3. Se dă gramatica:

```
E-> E + T | T
T-> T * F | F
F-> (E) | ct_num
```

Să se scrie acțiunile semantice care să calculeze valoarea expresiei aritmetice E.

$E \rightarrow E + T$ { E.val = E1.val + T.val }

$E \rightarrow T$ { E.val = T.val }

$T \rightarrow T * F$ { T.val = T1.val * F.val }

$T \rightarrow F$ { T.val = F.val }

$F \rightarrow E$ { F.val = E.val }

$F \rightarrow ct_num$ { F.val = ct_num.lexval }

4.

4. Scrie semantica operațională pentru următorul cod:
`a=2025; while(a>0){b=2025; while(b>0) {a--;b--}}`.
Imaginează un set de instrucțiuni de nivel scăzut și etichete pentru a scrie codul echivalent. Explică fiecare dintre instrucțiunile pe care le-ai utilizat.

```
a = 2025;
while (a>0) {
    b = 2025;
    while (b>0) {
        a--;
        b--;
    }
}
```

Variantă ca pe curs

```
a := 2025
b := 0
L1 :
    if a < 0 goto ENB
    b := 2025
L2 :
    if b <= 0 goto L1
    a := a - 1
    b := b - 1
    goto L2
ENB: HALT
```

Assembly

```
    MOV AX, 2025
L1: CMP AX, 0      ; compar a cu 0
    JLE ENB        ; a ≤ 0 => ieșim
    MOV BX, 2025
L2: CMP BX, 0
    JLE L1
    DEC AX
    DEC BX
    JMP L2
ENB: HLT sau NOOP
```

5.

5. Reduceți:
E1=(λg.(g λy.y) λb.(b b))
E2=(λf.(f (λg.(g λy.y) λb.(b b))) λa.(a a))

$E_1 = (\lambda g.(g\ \lambda y.y)\ \lambda b.(b\ b))$

Aplicăm λg argumentului $\lambda b.(b\ b)$

$\Rightarrow$ Înlocuim λg cu $\lambda b.(b\ b)$

$E_1 = \lambda b.(b\ b)\ \lambda y.y$

$E_1 = \lambda y.y\ \ \lambda y.y$ → identitate

$E_1 = \lambda y.y.$

$E_2 = (\lambda f.(f\ (\underbrace{\lambda g.(g\ \lambda y.y)\ \lambda b.(b\ b)}_{E_1}))\ \lambda a.(a\ a))$

$E_2 = (\lambda f.(f\ \lambda y.y)\ \lambda a.(a\ a))$

$E_2 = (\lambda a.(a\ a)\ \lambda y.y)$

$E_2 = (\lambda y.y\ \ \lambda y.y)$

$E_2 = \lambda y.y$